

■ Lab 14 — Terraform Import

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Créé une ressource AWS **manuellement** depuis la console
- Importé cette ressource dans le state Terraform
- Utilisé la **nouvelle fonctionnalité `import block`** (Terraform 1.5+)
- Généré automatiquement la configuration avec `-generate-config-out`

■ Étape 1 — Créer une ressource hors Terraform

Créez un Security Group manuellement depuis la CLI AWS :

```
# Créer le Security Group hors Terraform
SG_ID=$(aws ec2 create-security-group \
  --group-name "lab14-<votre-prenom>-manual-sg" \
  --description "SG créé manuellement - Lab 14" \
  --query 'GroupId' \
  --output text)

echo "Security Group ID : $SG_ID"

# Ajouter un tag
aws ec2 create-tags \
  --resources $SG_ID \
  --tags Key=Name,Value="lab14-<votre-prenom>-manual-sg" \
    Key=Lab,Value=lab14 \
    Key=Username,Value=<votre-prenom>
```

■ Étape 2 — Créer les fichiers

```
cd labs/lab14-import
```

```
`backend.tf`
```

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ ■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prénom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}

variable "sg_id" {
  description = "ID du Security Group à importer"
  type        = string
}
```

■ Partie A — Méthode classique : terraform import

Étape A1 — Déclarer la ressource dans main.tf

`main.tf`

```
resource "aws_security_group" "imported_sg" {
  name          = "lab14-${var.username}-manual-sg"
  description   = "SG créé manuellement - Lab 14"

  tags = {
    Name     = "lab14-${var.username}-manual-sg"
    Lab      = "lab14"
    Username = var.username
  }
}
```

Étape A2 — Importer la ressource dans le state

```
terraform init

terraform import \
  -var="username=<votre-prenom>" \
  -var="sg_id=$SG_ID" \
  aws_security_group.imported_sg $SG_ID
```

Résultat attendu :

```
Import successful!
The resources that were imported are shown above. These resources are now in your Terraform state.
```

Étape A3 — Vérifier l'import

```
terraform state list
terraform state show aws_security_group.imported_sg
terraform plan -var="username=<votre-prenom>" -var="sg_id=$SG_ID"
```

Résultat attendu : `No changes.` — la configuration correspond à la ressource importée.

■ Partie B — Nouvelle méthode : import block (Terraform 1.5+)

Étape B1 — Créer un second Security Group à importer

```
SG_ID2=$(aws ec2 create-security-group \
  --group-name "lab14-<votre-prenom>-manual-sg2" \
  --description "SG2 créé manuellement - Lab 14" \
  --query 'GroupId' \
  --output text)

echo "Security Group 2 ID : $SG_ID2"
```

Étape B2 — Ajouter le bloc import dans main.tf

```
import {
  to = aws_security_group.imported_sg2
  id = "<SG_ID2>" # Remplacez par l'ID réel
}
```

Étape B3 — Générer automatiquement la configuration

```
terraform plan \
  -var="username=<votre-prenom>" \
  -var="sg_id=$SG_ID" \
  -generate-config-out=generated_sg2.tf
```

Inspectez le fichier généré :

```
cat generated_sg2.tf
```

■ Terraform génère automatiquement le bloc `resource` correspondant à la ressource AWS — plus besoin de l'écrire manuellement !

Étape B4 — Appliquer l'import

```
terraform apply \
  -var="username=<votre-prenom>" \
  -var="sg_id=$SG_ID"
```

■ Étape 3 — Nettoyage

```
terraform destroy \  
-var="username=<votre-prenom>" \  
-var="sg_id=$SG_ID"
```

■ Validation du Lab

#	Critère	Validé
1	Security Group créé manuellement via AWS CLI	■
2	`terraform import` importe le SG dans le state	■
3	`terraform plan` retourne "No changes" après import	■
4	Bloc `import` + ` -generate-config-out` génère ` generated_sg2.tf`	■
5	`terraform apply` importe le 2ème SG avec la nouvelle méthode	■
6	`terraform destroy` supprime toutes les ressources	■

■■ Résolution des problèmes courants

Problème	Cause	Solution
`Resource already managed`	SG déjà dans le state	Vérifier avec `terraform state list`
`No changes` après import	Configuration ne correspond pas	Aligner `main.tf` avec la ressource réelle
`generate-config-out` échoue	Fichier déjà existant	Supprimer le fichier et relancer

■ ****Fin du Lab 14 — Vous gérez des ressources existantes avec Terraform !****

Félicitations — vous avez complété les 14 labs de la formation Terraform ! ■